EXP-7 Integrate a Vector Database with an LLM to build a Retrieval-Augmented Generation (RAG) system that answers questions based on external PDF documents.

AIM:

To build a Retrieval Augmented Generation (RAG) system that integrates a vector database with a language model to answer queries using external PDF documents.

DATASET:

Linkedin Profile PDF

PROCEDURE:

Step 1: Import Libraries

```
from langchain.document_loaders import PyPDFLoader from
langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.vectorstores import FAISS
from langchain.embeddings import OpenAIEmbeddings
from langchain.chains import RetrievalQA from
langchain.llms import OpenAI
```

Step 2: Load PDF

```
from google.colab import files
uploaded = files.upload()
loader = PyPDFLoader("your_file.pdf")
documents = loader.load()
```

Step 3: Text Chunking

```
text_splitter =
   RecursiveCharacterTextSplitter( chunk_size=500,
   chunk_overlap=50
) texts =

text_splitter.split_documents(documents)
```

Step 4: Create Embeddings import os
os.environ["OPENAI_API_KEY"] = "your_api_key_here"
embeddings = OpenAIEmbeddings()

Step 5: Store in Vector Database (FAISS) vector_db = FAISS.from_documents(texts, embeddings)

Step 6: Build Retrieval System
retriever = vector_db.as_retriever()

Step 7: Create RAG Chain
qa_chain =
RetrievalQA.from_chain_type(llm=OpenAI(),
retriever=retriever,
return_source_documents=True
)

Step 8: Ask Questions query = "What are the key skills mentioned in the document?" result = qa_chain(query)
print("Answer:\n", result["result"])

OUTPUT:

The document highlights skills such as Python, Machine Learning, Data Analysis, and Deep Learning.

RESULT:

The RAG system successfully retrieved relevant content from the PDF and generated accurate, context aware answers to user queries.